PROJECT CASE STUDY

EchoNote

An AI meeting-intelligence application that turns recorded conversations into searchable, speaker-attributed transcripts, structured summaries, and connected follow-up work.

SYSTEM	SCOPE	STATUS
Full-Stack Web Application	**Frontend · Backend · Audio · AI · Collaboration**	**Deployed**

01 The Problem

Meetings create decisions and commitments, but the record of what happened is often scattered across audio files, personal notes, and task trackers.

Replaying a recording to find one detail is slow. Writing notes while participating divides attention. Even when a summary exists, the speaker context, original discussion, and follow-up work can become disconnected.

> EchoNote brings the recording, transcript, summary, decisions, and extracted action items into one meeting record — so people can revisit the discussion and carry its context into the work that follows.

02 What Was Built

A connected application spanning meeting capture, AI processing, personal follow-through, and team collaboration.

CAPTURE & MEETING LIBRARY

Record in the browser or upload audio. Organize meetings by title and category, search the library, and follow processing status.

MEETING INTELLIGENCE

Prepare audio, transcribe with speaker attribution, enrich the text, and generate structured summaries, decisions, and action candidates.

FOLLOW-THROUGH WORKSPACE

Keep generated tasks linked to the source meeting. Review ownership and deadlines, update task status, and revisit supporting context.

SHARING & COLLABORATION

Share meeting summaries, organize meetings into workspaces, and collaborate with owner, editor, and viewer access roles.

03 How It Works

The browser accepts the recording and then tracks asynchronous processing through the API. An Express backend coordinates a local worker, Python audio and language tools, external AI providers, and PostgreSQL persistence.

React browser → Express API → In-process FIFO worker

Audio preparation → Deepgram → spaCy → Groq summaries

PostgreSQL / Prisma · Supabase or local audio storage

Capture & admission

An authenticated upload carries the audio and meeting metadata. The API checks file type, size, and duration, creates a user-owned meeting, and returns its ID before analysis finishes.

Audio & transcription

Python prepares mono, 16 kHz PCM WAV. Deepgram produces punctuated transcript text, timestamped speaker segments, confidence information, and supporting language signals.

Enrichment & structured outputs

Local spaCy analysis extracts entities, relations, questions, and action-language signals. A Groq-backed language-model service uses the transcript and available features to propose a summary, decisions, action items, and next steps.

Persistence & follow-through

The transcript, summary, and processing state stay on the meeting. Action items become linked task records. Optional Jira synchronization and completion notifications connect the result to existing workflows.

04 Key Features

- **Browser recording & audio upload** — capture microphone audio or submit a supported file, with meeting metadata and validation.
- **Searchable meeting library** — find and filter meetings, inspect their status, and return to the original discussion.
- **Speaker-attributed transcripts** — review timestamped segments and refine speaker mappings for clearer context.
- **AI-generated meeting intelligence** — review executive summaries, key decisions, topics, and suggested next steps.
- **Connected action tracking** — turn extracted actions into meeting-linked tasks with ownership, deadlines, priorities, and TODO / IN_PROGRESS / DONE states.
- **Calendar context** — retrieve Google Calendar events and associate meeting metadata.
- **Sharing & team workspaces** — publish summary links and organize workspace meetings with role-based membership.
- **Realtime collaboration** — use Liveblocks-backed editing authorized against meeting ownership or workspace role.
- **Optional workflow integrations** — synchronize Jira work and send completion updates through email, web push, or a configured Slack webhook.

05 Technical Approach

The implementation separates browser interaction, API requests, background processing, structured data, and provider integrations. The following stack supports those boundaries.

LAYER	STACK	PURPOSE
Frontend	React 18, React Router, HeroUI, Tailwind, RecordRTC	Capture, meeting review, tasks, and workspaces
API & identity	Node.js, Express, JWT, Google OAuth	Authentication, validation, and domain operations
Processing	In-process FIFO worker; Python audio tools; spaCy	Audio preparation and local language enrichment
AI services	Deepgram; Groq-backed language-model service	Speaker transcription, summaries, action extraction
Persistence	PostgreSQL, Prisma; Supabase or local files	Meeting and task records; processed audio
Collaboration	Liveblocks; Google Calendar; Jira, Slack, email, Web Push	Shared editing, context, and optional follow-through

Design decisions that matter

Meeting as the durable context. Audio, transcript, generated outputs, and tasks remain associated rather than becoming disconnected exports.

Visible asynchronous work. Persisted stages and status polling keep the browser informed without holding the upload request open. Queue retries use backoff; NLP failure can degrade to empty enrichment.

Explicit access boundaries. JWT authentication, ownership checks, and workspace roles gate protected resources and Liveblocks authorization.

06 Value Delivered

From a recording alone to a connected record of discussion and follow-through:

Searchable	Structured	Connected
MEETING MEMORY	DECISIONS & ACTIONS	TEAM FOLLOW-THROUGH

- ✓ **Less manual reconstruction.** — A transcript and structured summary provide a reviewable starting point instead of requiring every detail to be recovered from the audio.
- ✓ **Context for the work.** — Meeting-linked action items keep proposed commitments connected to the discussion that produced them.
- ✓ **A shared review surface.** — Summary links, workspaces, and collaboration let people return to the same meeting context rather than circulate separate copies.
- ✓ **Follow-through in existing tools.** — Optional Jira and notification integrations bring meeting outputs into the channels where work is already coordinated.

EchoNote — a full-stack meeting-intelligence application combining audio processing, speaker-aware transcription, AI summarization, and collaborative follow-through in one connected workflow.